

۱- نادرست - معماری نرم افزار مجموعه ای از تصمیمات کلان و مهم تمرکز دارد که بر اساس هزینه تغییر سنجیده می شوند.
در واقع معماری خوب به دنبال ~~آن~~ به تعویق انداختن تصمیمات مربوط به جزئیات تا جایی ممکن است. در حالی که معماری شکل کلی نرم افزار و سیستم تعیین می کند، طراحی به جزئیات پیاده سازی می پردازد.

۲- نادرست - بر اساس اصل CCP باید کلاس هایی که به دلایل مشابه و در زمان های مشابه تغییر می کنند را در یک پکیج قرار داد. این اصل صراحتاً اعلام می کند کلاس هایی که به دلیل متفاوت تغییر می کنند باید از هم جدا باشند. اصل CCP بر محلت تغییر تمرکز دارد تا هزینه های نگهداری و باز نشر کاهش یابد.

۳- درست - الگوی معماری broker برای سیستم های توزیع شده و Event driven که نیاز به ارتباط Async دارند بسیار مناسب است. در یک برنامه کارگزاری بوسیله حجم بالایی از رویدادها و نیاز به هماهنگی بین سرویس های مختلف روبه روست، الگوی broker می تواند با تمرکززدایی و مدیریت پیام ها، ارتباط بین اجزاء را تسهیل کند.

۴- نادرست - Entity ها باید کاملاً مستقل از دغدغه های خارجی مانند پایگاه داده یا UI باشند. آن ها در ~~درون~~ درونی ترین لایه قرار دارند و نباید هیچ وابستگی به لایه های بیرونی داشته باشند. وابستگی مستقیم به DB یا UI اصل استقلال منطقی کسب و کار از تکنولوژی را نقض می کند و قابلیت تست و تغییر سیستم را کاهش می دهد.

۵- نادرست - یک Core subdomain بخشی از کسب و کار است که به شرکت مزیت رقابتی نسبت به رقیب می دهد و استراتژی اصلی برای ارائه خدمات بهتر یا به حد اکثر رساندن سود است. لزوماً اینطور نیست که کاربر بابت آن مستقیم پول بدهد. مثلاً الگوریتم Ranking در گوگل یک Core subdomain حیاتی است اما کاربر بخاطر آن پول نمی دهد و رایگان است.

۶- درست - در DPP و معاری Clean ، ریاضیتوی یک انتزاع فراهم می کند که دسترسی به اشیاء دامنه را شبیه به یک مجموعه در حافظه جاوه می دهد هدف اصلی آن پنهان کردن جزئیات پیچیده مربوط به پایگاه داده و زیرساخت از لایه دامنه است .

کلاس `orderprocessor` و `paymentService` هر دو تاریخ و محاسبه می کنند و لاگ می کنند. این منطق تکراری است که توسط هر دو کلاس پیاده سازی شده.

(الف)
- نقض `SRP`: `orderprocessor` همزمان چند کار انجام می دهد. محاسباتی سفارش + محاسبه تاریخ + لاگ کردن یک کلاس باید یک دلیل برای تغییر داشته باشد. این کلاس به ۳ دلیل ممکن است تغییر کند. در مورد `paymentService` هم سرویس پرداخت نباید تاریخ حساب کند و لاگ کند.

- نقض `DIP`: کلاس `order` مستقیماً به کلاس های لاگ و پرداخت وابسته است. نه به یک `Interface`. ماژول سطح بالایی نباید به ماژول سطح پایین وابسته باشد. هر دو باید به انتزاع وابسته باشند.

- نقض `DRY` و `OCP`: منطق محاسبه تاریخ و لاگ در دو کلاس پیچیده شده. اگر بخواهیم قیمت تاریخ را عوض کنیم باید چند کلاس را عوض کنیم. برای توسعه باز و برای تغییر سبک نیستیم! (نقض `OCP`)

(ب)
`Coupling` بالا است! : مفهوم زمان لاگ داخل دو کلاس است پس این دو کلاس به هم گره خورده اند. تغییر در یکی، دیگری را فرا می گیرد. کلاس `orderproc` به پیاده سازی متد `paymentService` و `logger` به صورت سخت و سفت وابسته است.

`Cohesion` پایین است! : کلاس `orderproc` کارهای نامرتبط را در خود جمع کرده است. این یعنی هم بستگی داخل این کلاس پایین است. منطق ترید که باید یک جا منسجم باشد بین کلاس ها پخش شده! کماهی مرتبط به یک مسئولیت گناهم تجمیع نشده اند!

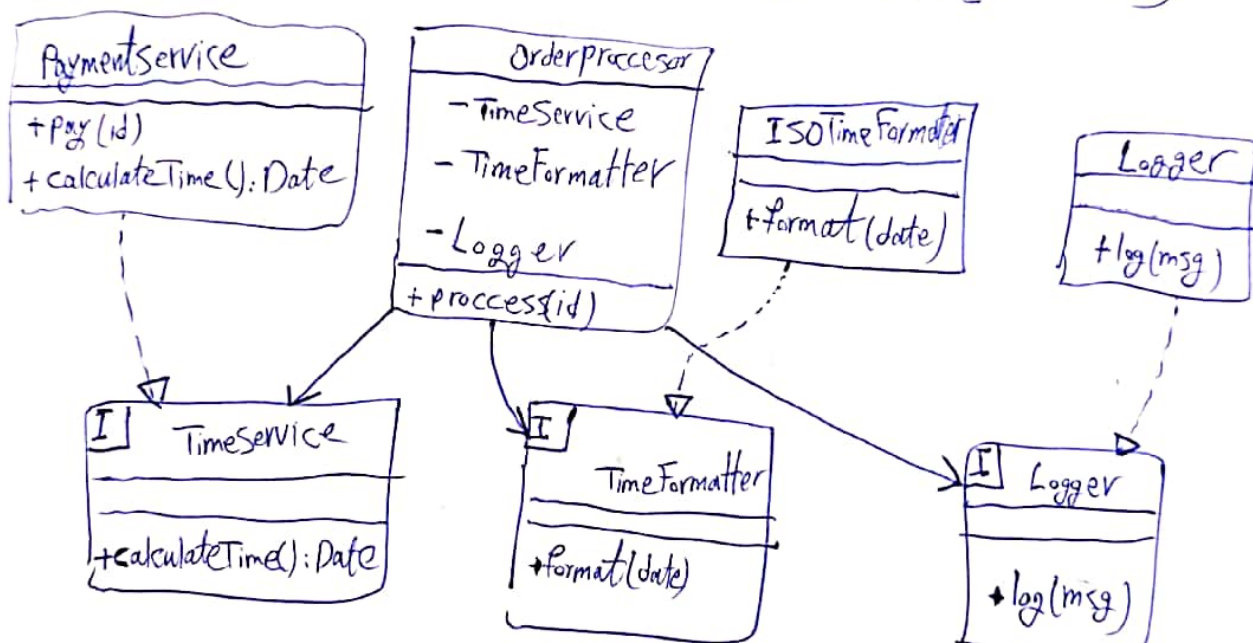
۱) اقسام ها: ۱) استخراج مسئولیت قالب بندی تبلیغ در یک اینترفیس TimeFormatter بایجاد سازی واحد.

این هم DRY اصل می باشد هم بک فوتهای متفاوت رو

۲) متمرکز کردن منطق محاسبه زمان پشت یک Interface به نام ServiceTime تا تاریخ شروع و پایان معرعه بکسان و دارای مالک مشخص باشند و Coupling را کاهش دهد.

۳) وابستگی به انتزاع به جای وابستگی مستقیم به کلاس های واحد.

۴) تک مسئولیتی کردن کلاس ها.



الف) ریشه اصلی این مشکل بحفت شکی نیست (Tight Coupling) بین منطق کسب و کار و جزئیات پیاده سازی (فریم ورک و سرویس خارجی) است. وقتی منطق اصلی سیستم در همان جایی پیاده سازی می شود که به فریم ورک دسترسی دارد، کدها با هم مخلوط شده و به پیره Big ball of mud رخ می دهد، وضعیتی که در آن افزا چنان به هم گره خورده اند که حتی تغییرات جزئی، هفته ها زمان برده و ریسک های بزرگی به همراه دارند. در واقع این طراحی فاقد مرزبندی های معماری درست است و سیاست های سطح بالا به جای مستقل بودن با جزئیات پیاده سازی وابسته شده اند.

ب) ریفتکتور به Clean به این دلیل توصیه می شود:

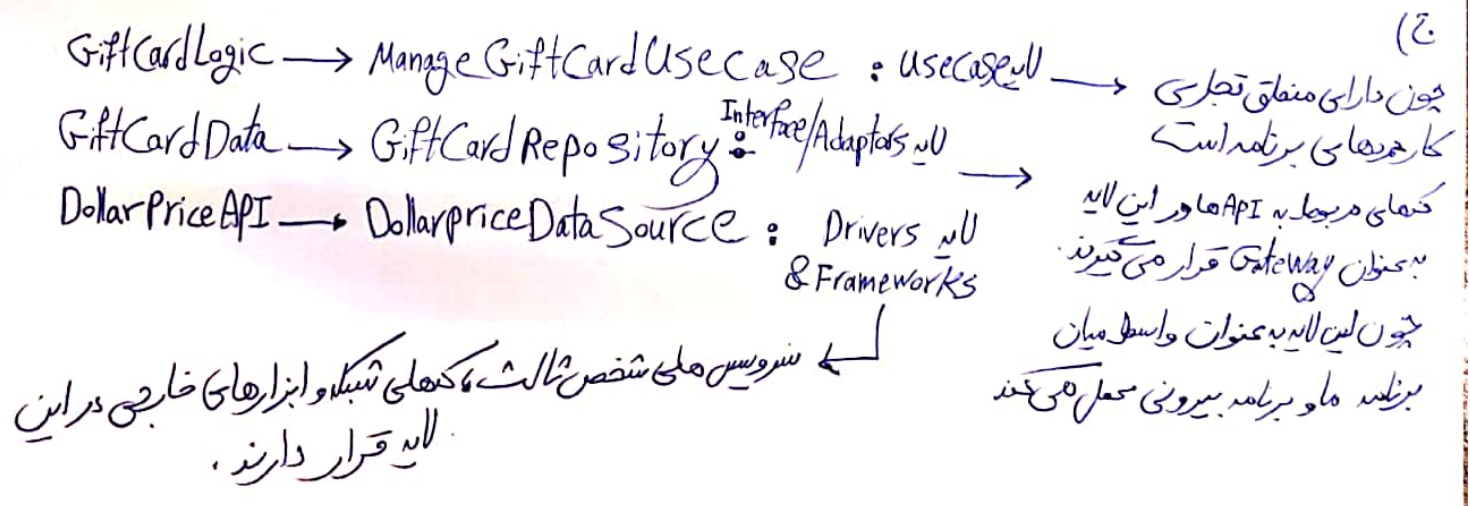
- تست پذیری بالا: با انزوله کردن منطق بیزینس از فریم ورک ها و سیستم های خارجی، می توان کدها را بدون نیاز به دیپایس یا سرور واقعی تست کرد. همچنین separation of concerns هم دارد.

- قابلیت نگهداری و تغییر: این معماری اجازه می دهد تکنولوژی ها (مثلاً تغییر دیپایس یا فریم ورک) را با کمترین تأثیر بر هسته مرکزی سیستم عوض کنید.

- استقلال از فریم ورک: بیزینس شما به ابزارهای بیرونی وابسته نخواهد بود و فریم ورک صرفاً به عنوان یک ابزار در لایه های بیرونی قرار می گیرد.

- توسعه مولزی: تیم های مختلف می توانند به دلیل جداسازی لایه ها و استفاده از اینترفیس ها، به صورت مستقل بر روی بخش های مختلف کار کنند.

ج)



برای تکمیل ه لایه Controller می توان کلاسی تعریف کرد که درخواست های ورودی کاربر مثلاً از فرانت را گرفته و به usecase پاس می دهد. نام کلاس: GiftCardController

مثل قرارگیری: لایه Interface & Adaptors که وظیفه آن دریافت درخواست خرید یا بررسی کارت هدیه از سمت کاربر و هدایت آن به GiftCardUsecase است.

(د) استفاده از Dependency Injection (تزریق وابستگی) باعث می شود کلاس ها دیگر خودشان مسئول

ساختن و مدیریت اشیا مورد نیازشان نباشند! این وابستگی ها از بیرون به آن ها تزریق می شود این کار با جداسازی فرآیند ساخت شیء از منطق اجرایی آن، Tight Coupling را از بین می برد.

در نتیجه کدهای تکراری مقداردهی ها حذف شده و کد بیس ما به شدت منظم تر، انعطاف پذیرتر و تست پذیرتر می شود.

۱- وردپرس: الگوی میکروکنترل / پلاگین: این معماری برای سیستم‌هایی که نیاز به توسعه پذیری و شخصی سازی بالا دارند بسیار مناسب است. در وردپرس یک Core وجود دارد و قابلیت‌های مختلف توسط پلاگین‌هایی که توسط افراد مختلف در دنیا نوشته می‌شوند به سیستم اضافه یا از آن حذف می‌گردد بدون اینکه نیاز باشد هر بار هسته اصلی تغییر کند.

۲- سرویس Match Making: الگوی Broker: این الگو برای سیستم‌های توزیع شده و Event driven که در آن اجزا مستقیماً با هم در ارتباط نیستند و نیاز به یک هماهنگ کننده مرکزی دارند، استفاده می‌شود. این سرویس باید درخواست‌های بازیکن‌های مختلف را دریافت کرده و پس از پیکار کردن صرف مناسب، ارتباط بین آن‌ها را برقرار کند.

۳- نسخه موبایل تلگرام: الگوی MVVM: این الگو برای اپلیکیشن‌های native که سمت کلاینت هستند و دارای رابط کاربری پیچیده و data-driven هستند مناسب است. این اپ‌ها نیاز به به روز رسانی دائمی state دارند با جداسازی منطق نمایش از منطق کسب و کار در لایه View/Model بهره می‌بریم. همچنین الگوی CQRS برای جداسازی دیتابیس Read از Write توصیه می‌شود چرا که حجم خواندن بسیار زیاد است و نوشتن بسیار زیاد است.

۴- بک اند CRM: معماری میکرو سرویس: برای یک سازمان بسیار بزرگ، سیستم CRM شامل بخش‌های متنوعی است مثل مالی/فروش/بسته بندی و... هر کدام می‌توانند به عنوان یک سرویس مستقل با قابلیت استقرار و توسعه مجزا توسط تیم‌های مختلف مدیریت شوند. همچنین در دل هر سرویس استفاده از معماری Clean یا هگزagonal برای مدیریت خوانش پیچیده کسب و کار توصیه می‌شود تا منطق اصلی سیستم از جزئیات تکنولوژی جدا بماند.

مدل سازی هر دو مفهوم متفاوت **با یک کلاس واحد** (Account) مصداق بارز مدل های ناسازگار است.
این وضعیت به دلیل زیر مدل دامنه را تعریف می کند:

وقتی یک کلاس در تیم های مختلف معانی متفاوتی دارد، فرآیند ترجمه دانش بین متخصصین دامنه و تیم فنی دچار خطا می شود و اصطلاحات در این میان گم می شوند.

پیشگیری بیش از حد: تلاش برای ایجاد یک کلاس واحد که همه نیازها را پوشش دهد، باعث می شود آن کلاس در هیچ کدام از حوزه ها کارآمد نباشد. این کلاس باید همزمان توانایی کسب و کار در حوزه به مشتری و قوانین پیچیده حسابداری داخلی را مدیریت کند که باعث افزایش شدید هزینه نگهداری و احتمال بروز باگ می شود.

تداخل قوانین کسب و کار: تغییر در منطق حسابداری مالی ممکن است ناخواسته بر منطق حساب مشتری تاثیر بگذارد، زیرا هر دو به یک قطعه کد وابسته هستند.

برای شناسایی مرزها، باید از روش های تحلیل استراتژیک DDD استفاده کرد:

- تحلیل زبان: شناسایی تقاطعی که یک واژه معانی متفاوتی پیدا می کند. این تفاوت معنایی نشان دهنده یک خط شکست زبانی است که مرز بین دو حوزه را مشخص می کند.

- مدل های ذهنی: باید بررسی کرد که متخصصین هر بخش چگونه به مسئله نگاه می کنند. برای تیم محصول، Account یک موجودیت برای ترکانش مشتری است، اما برای تیم مالی، بخشی از ترازنامه و دفتر کل است.

- استفاده از event storming: با ترسیم رویدادهای کسب و کار، می توان خوشه هایی از رویدادها که به هم مرتبط هستند را شناسایی کرد. این خوشه ها معمولاً مرزهای زیردامنه و کانتکست ها را نشان می دهند.
پیشنهادات برای نافرگذاری، مدل سازی و همکاری با متخصصین دامنه:

- تعریف bounded context های مجزا: سیستم باید به کانتکست محصور جلگانه تقسیم شود. مثلاً Accounting و Product / banking و هر کانتکست مدل و زبان مشترک خاص خود را خواهد داشت.

ادامه سوال که

- تفسیر در محل سازی و نام گذاری: هر به جای استفاده از یک کلاس Account عمومی، در هر کانتکست از نام دقیق و مورد استفاده متخصصین آن بخش استفاده کنید. مثلاً در کانتکست محصول از Customer Account و در کانتکست مالی از Ledger Account استفاده شود. این کار باعث می شود که حد دقیقاً باریان متخصصین مطابقت داشته باشد.

- تدوین Glossary: ایجاد یک مستند یا ویکی که در آن اصطلاحات هر کانتکست به طور دقیق تعریف شده باشد. این واژه نامه باید همواره به روز رسانی و توسط کل تیم (فنی و تجاری استفاده شود)

- تعامل مستقیم با Domain Expert: متخصصین دانش باید مستقیم در فرآیند فرموله سازی زبان مشترک

داخل باشند تا اطمینان حاصل شود که مفاهیم ذهنی آن ها به درستی در کد فیلکت شده است.

۱- جریان رویه‌های دامنه و خط زمانی:

در استپ ۱ و ۲ کارگاه event storming رویه‌ها را به ترتیب وقوع ترسیم می‌کنیم: (Happy Path و مسیرهای جایگزین)

Pricing Policy Approve: تصویب قانون جدید قیمت گذاری توسط مدیران

Grandfathering Rules Defined: تعیین قوانین برخورد با قراردادهای قبلی (مثلاً قیمت قدیم برای مشتریان قدیمی ثابت بماند)

Manual Discount Validated: بازبینی تخفیف‌های دستی معتبر توسط تیم فروش

Subscription Renew Triggered: سررسید تمدید اشتراک یک مشتری

Plan Upgrade Request: درخواست مشتری برای ارتقای پلن

Price Calculated: محاسبه قیمت نهایی با در نظر گرفتن اولویت‌ها (قانون جدید، قرارداد قبلی یا تخفیف خاص)

Invoice Issued: صدور فاکتور نهایی

۲- تصمیمات و تعاملات بین تیمی:

در استپ ۳، Commands (آی) و Actors (زرت) مشخص می‌شوند.

محصول: فرمان Define Pricing Strategy صادر می‌کند تا مدل‌های جدید اشتراک و ارتقا با استراتژی شرکت همسو شود.

مالی: فرمان Apply Grandfathering Policy صادر می‌کند تا مشخص شود کدام قراردادها می‌توانند از افزایش قیمت معاف باشند.

فروش: فرمان override price manually را برای اعمال یا تمدید تخفیف‌های خاص صادر می‌کند.

فنی: وظیفه بهره‌سازی Consistency Enforcement را دارد تا مطمئن شود قوانین قیمت گذاری در تمام سطوح سیستم (تدوین، ارتقا و صدور فاکتور) به درستی اعمال می‌شوند.

۳- پالیسی ها و اتوماسیون (purple sticky notes): در استیج ۶، پالیسی ها به عنوان اگر سه آنگاه سه عمل می کنند.

Policy: اگر مشتری دارای قرارداد معتبر قبلی است، قیمت جدید را نادیده بگیر و طبق قرارداد قبلی محاسبه کن.
Policy: اگر درخواست ارتقا وجود دارد، مابین تفاوت را بر اساس قیمت جدید محاسبه کن اما تخفیف مالی دستی قبلی را بررسی کن.

۴- جلوگیری از صدور فاکتور اشتباه: (Aggregate و Invariants)

برای جلوگیری از خطا، مدل شما باید از مفاهیم Tactical DDD استفاده کند:

Aggregate Root (subscription): موجودیت اشتراک به عنوان یک Aggregate عمل کرده و تمامی قوانین مربوط به قیمت، تخفیف و قرارداد را در مرز خود محصور می کند. این تجزیه اجازه نمی دهد تغییری در سیستم رخ دهد که Invariants را نقض کند. (قوانین ثابت کسب و کار)

business Invariants: قانون «قراردادهای فیکس نباید تحت تأثیر تغییر قیمت سراسری قرار بگیرند» به عنوان یک قانون حیاتی و مهم در داخل موجودیت قرار می گیرد.

Event sourcing (dimension of time): اگر از این الگو استفاده شود، سیستم تاریخچه کامل تمام تخفیف ها و تغییرات قرارداد را دارد. بنابراین هنگام صدور فاکتور، سیستم با بازخوانی رویدادهای گذشته، وضعیت دقیق قیمت مشتری را در لحظه T محاسبه می کند و احتمال نادیده گرفتن یک تخفیف قدیمی یا قرارداد خاص به صفر می رسد.

bounded context: جداسازی کانتکست فروش از صدور صورت حساب باعث می شود پیچیدگی های مذاکرات فروش، مدل دقیق حسابداری فاکتور را آلوده نکند و محاسبات مالی در یک محیط کنترل شده انجام شود.

در معماری Clean، Gateway یک اینترفیس است که دسترسی به منابع خارجی مانند دیتابیس، API های خارجی یا فایل سیستم را پشت یک مرز مشخص پنهان / محصور می کند. وظیفه اصلی آن جداسازی یوزکیس ها از جزئیات فنی و معکوس کردن وابستگی هاست، به این معنا که لایه داخلی (یوزکیس) قرارداد را تعریف می کند و لایه خارجی (Infrastructure) آن را اجرا می کند تا مناطق بیرونش درگیر پیچیدگی های تکنولوژی نشود.

از سوی دیگر، data mapper یک مترجم است که میان ریف های خام دیتابیس و اشیا دامنه (entities) ارتباط برقرار می کند. کارکرد اصلی آن این است که موجودیت های دهنه را مستقل از پایگاه داده نگه دارد تا آن ها فقط بر رفتار و قوانین کسب و کار تمرکز کنند و از نحوه ذخیره سازی خود اطلاعی نداشته باشند. در این حالت لایه منطبق برنامه تمیز از ساختارهای بیرونی باقی می ماند.

مثال:

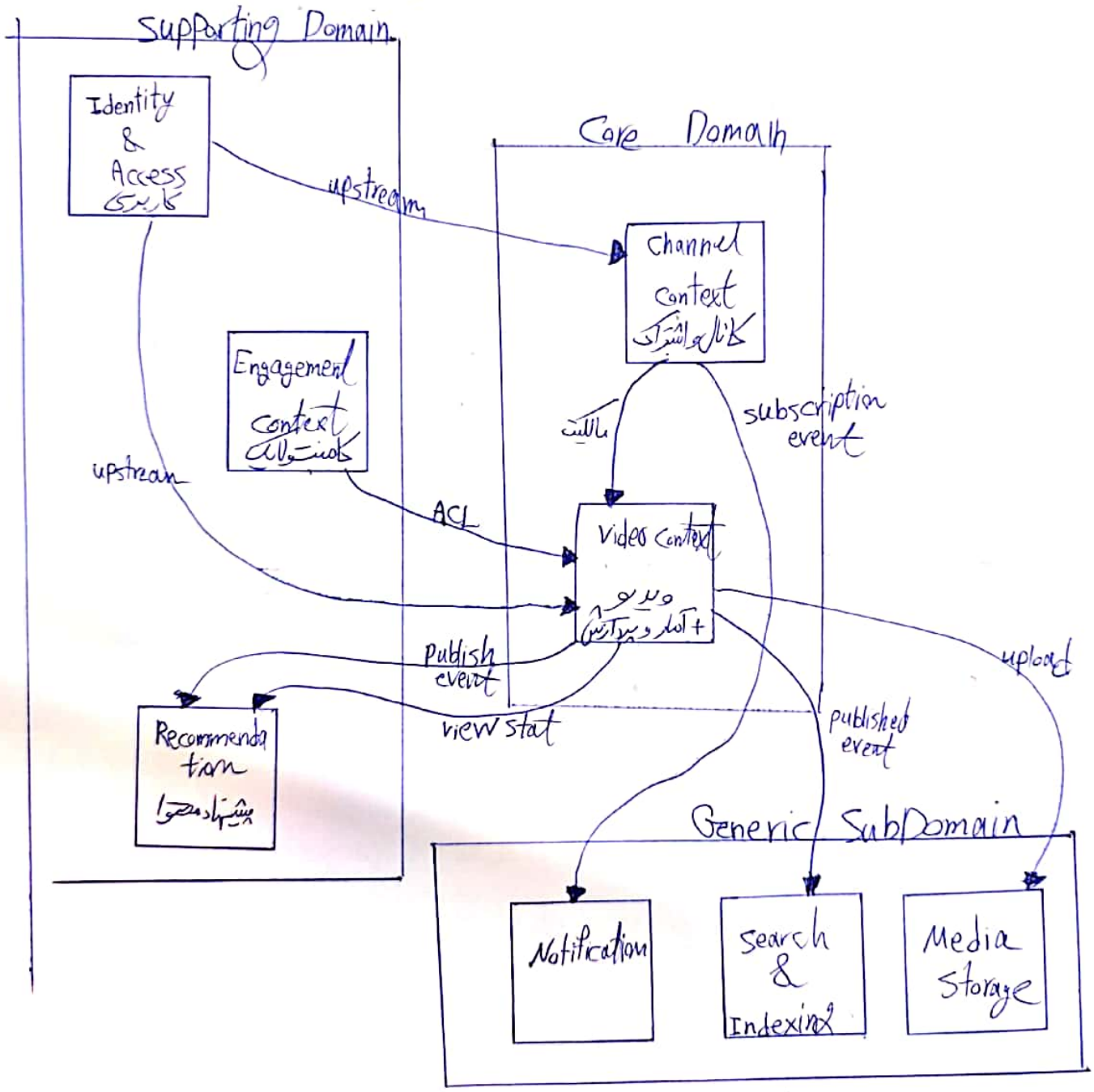
در یک سامانه ثبت سفارش، یوزکیس خرید به یک OrderGateway وابسته است که صرفاً دستور Save(order) را صادر می کند بدون اینکه بداند دیتا کجا ذخیره می شود. در لایه زیرساخت، ما پیاده سازی این گیت وی از یک orderMapper استفاده می کند تا شیء سفارش را که دارای متدهای محاسباتی است، به یک سطر ساده با ستون های مشخص برای ذخیره در دیتابیس تبدیل کند. این کار باعث می شود اگر روزی تصمیم بگیریم به جایی از SQL از Mongo استفاده کنیم، فقط پیاده سازی Gateway و Mapper تغییر کند و Core سیستم که یوزکیس و entity است کاملاً بدون تغییر باقی بماند. این مثال را می توان برعکس هم زد. مثلاً یک داده خام را بیک شیء قابل استفاده و تمیز در لایه درونی برنامه تبدیل کرد.

بہ دلائل اصلی سوال پر محدود تقسیم ہو سکتا ہے : bounded context

① Identity + Access context

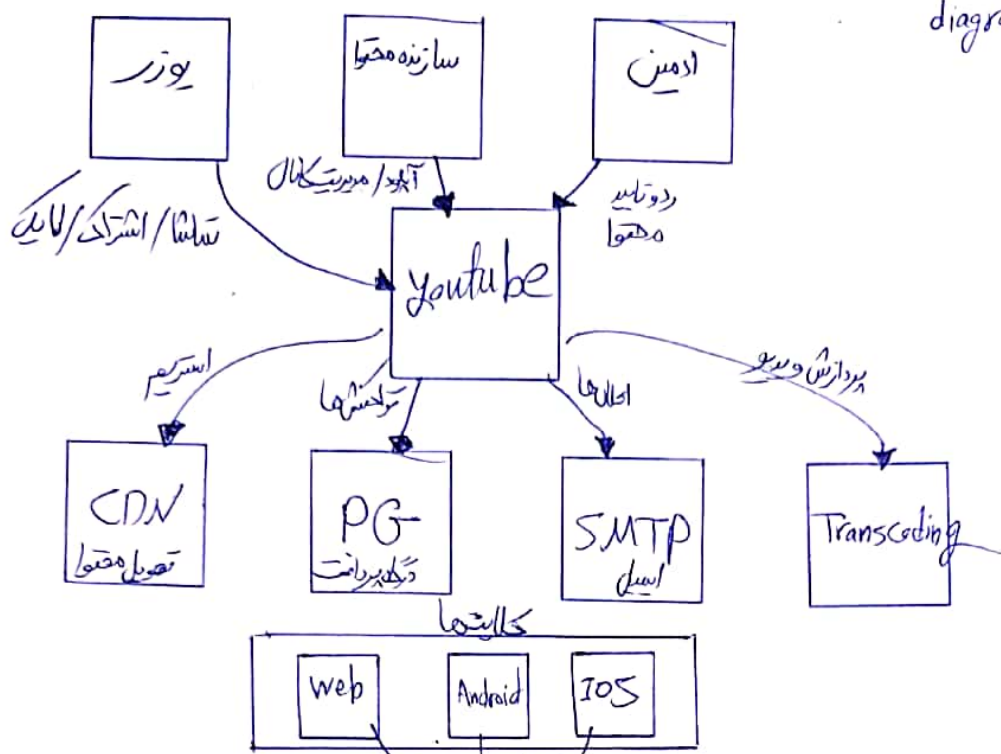
② Video context

③ channel context

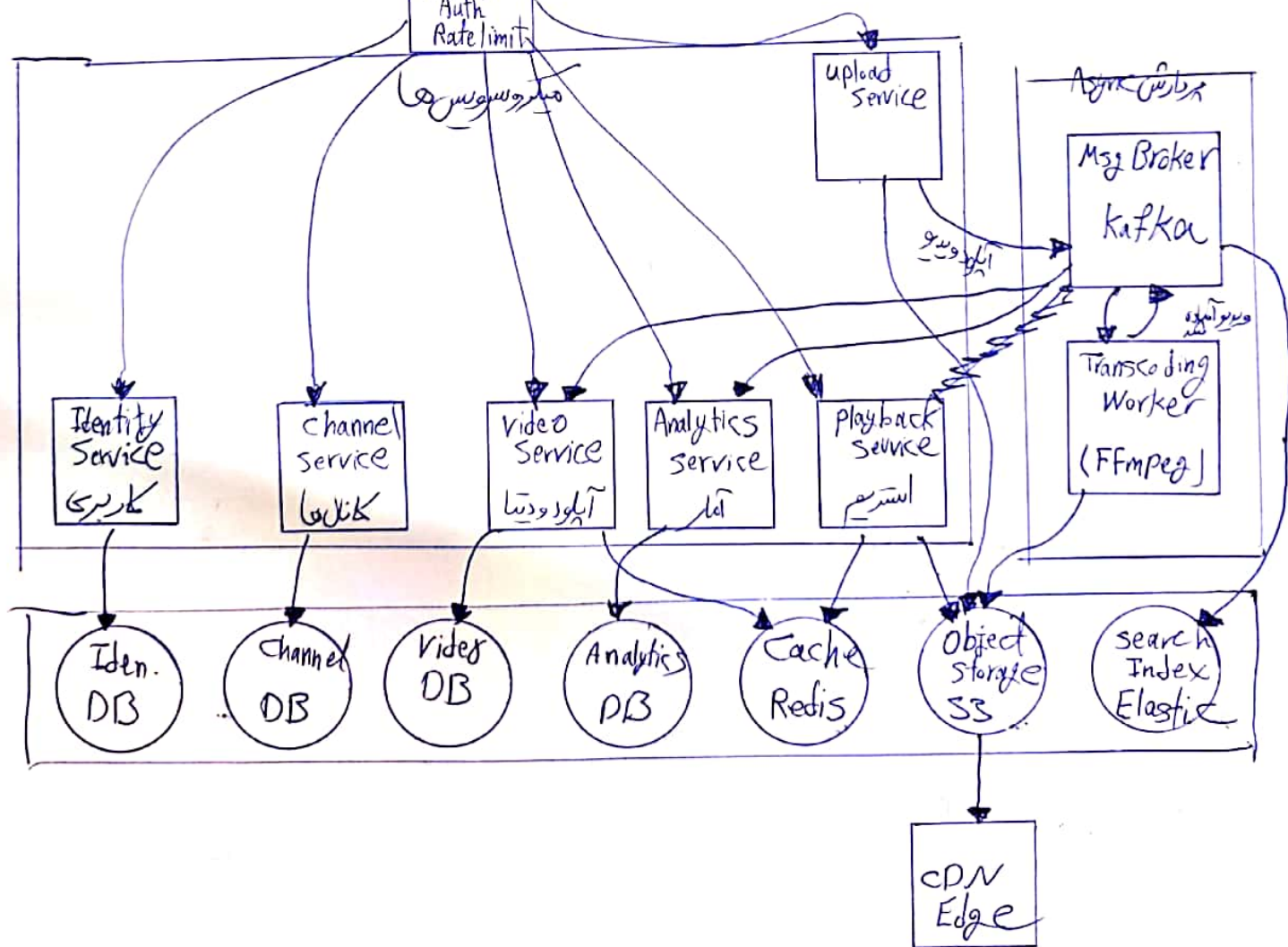


نوٹ: Video context و Channel context کا حصہ ہیں۔ چونکہ انہیں رابطہ قائم ہے۔

System context - C4 diagram

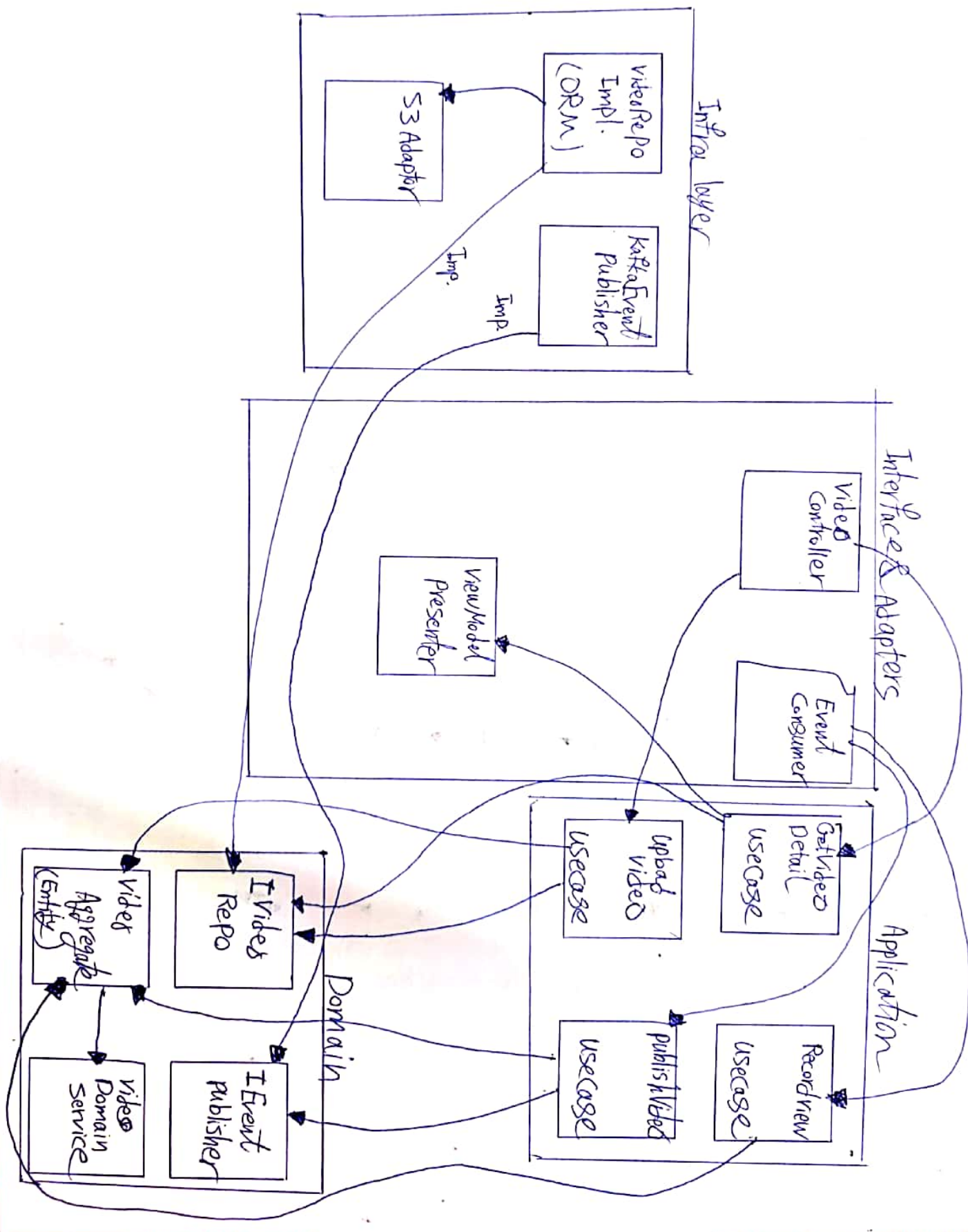


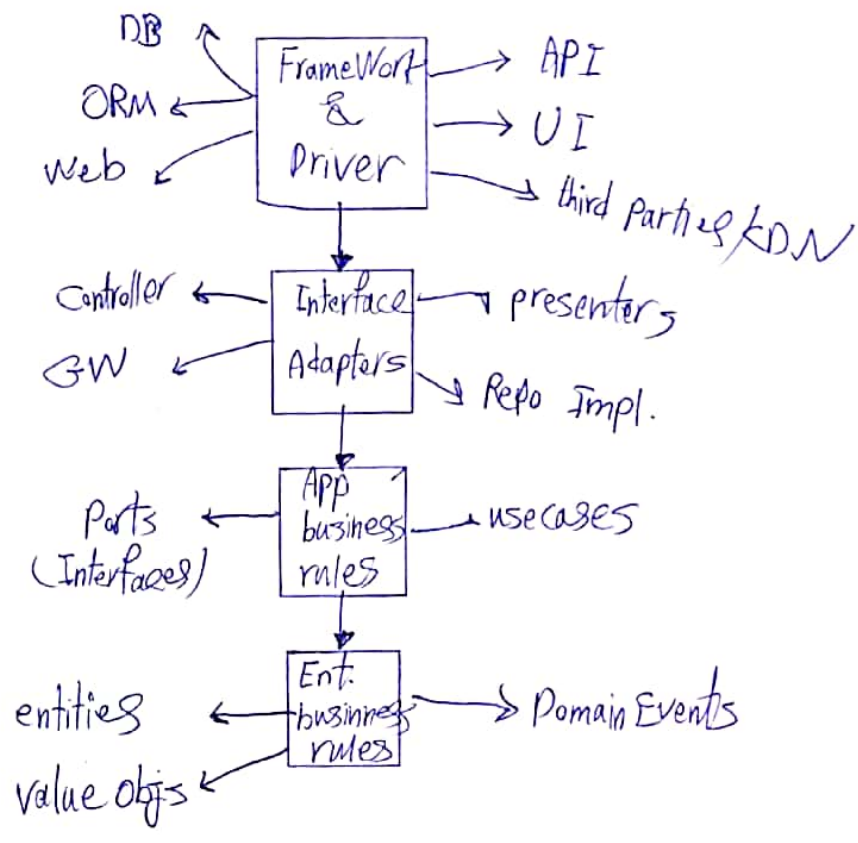
Container diagram - C4 diagram



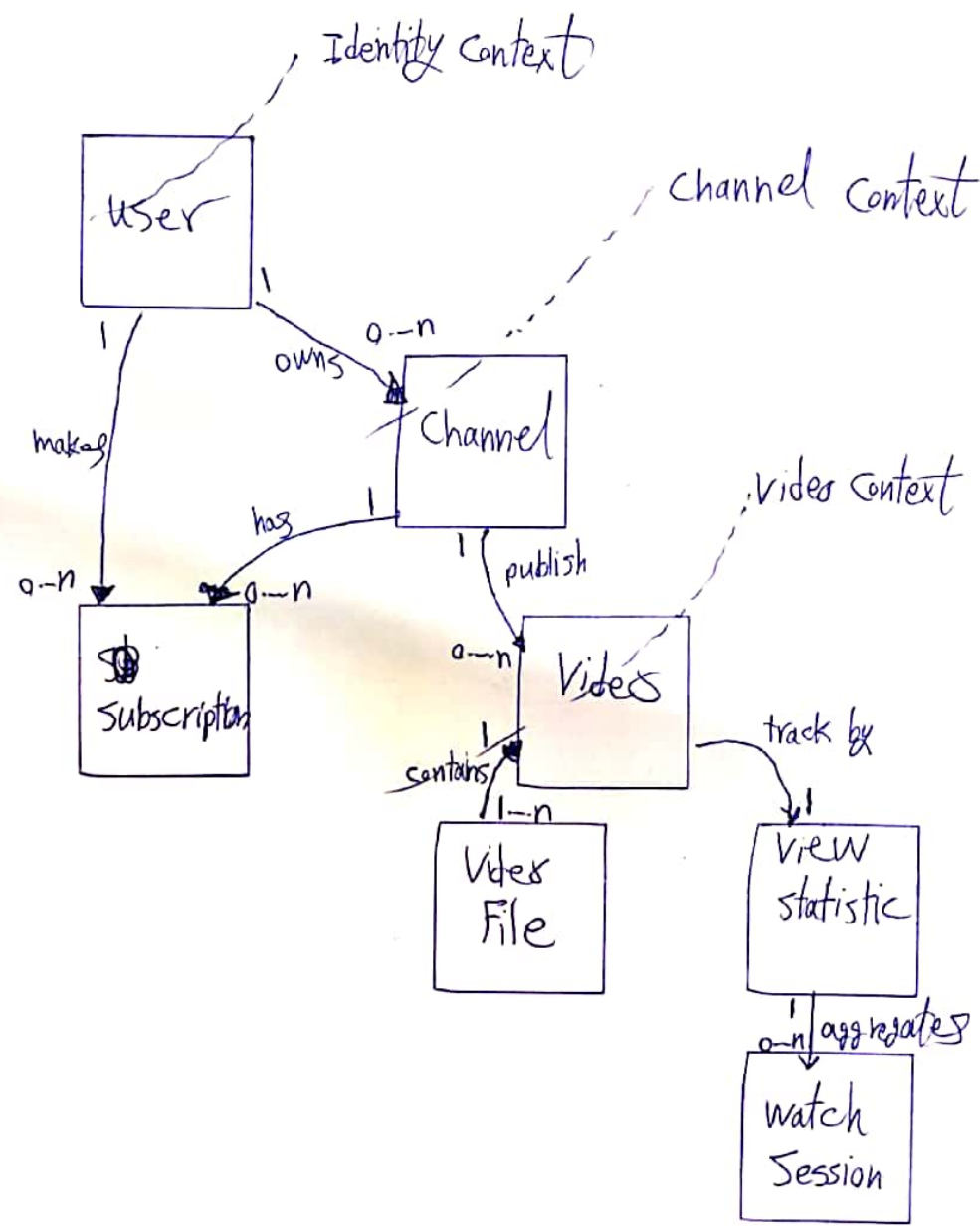
مدل C4 - Component Diagram (Video Service درون)

در این سرویس Clean معماری است

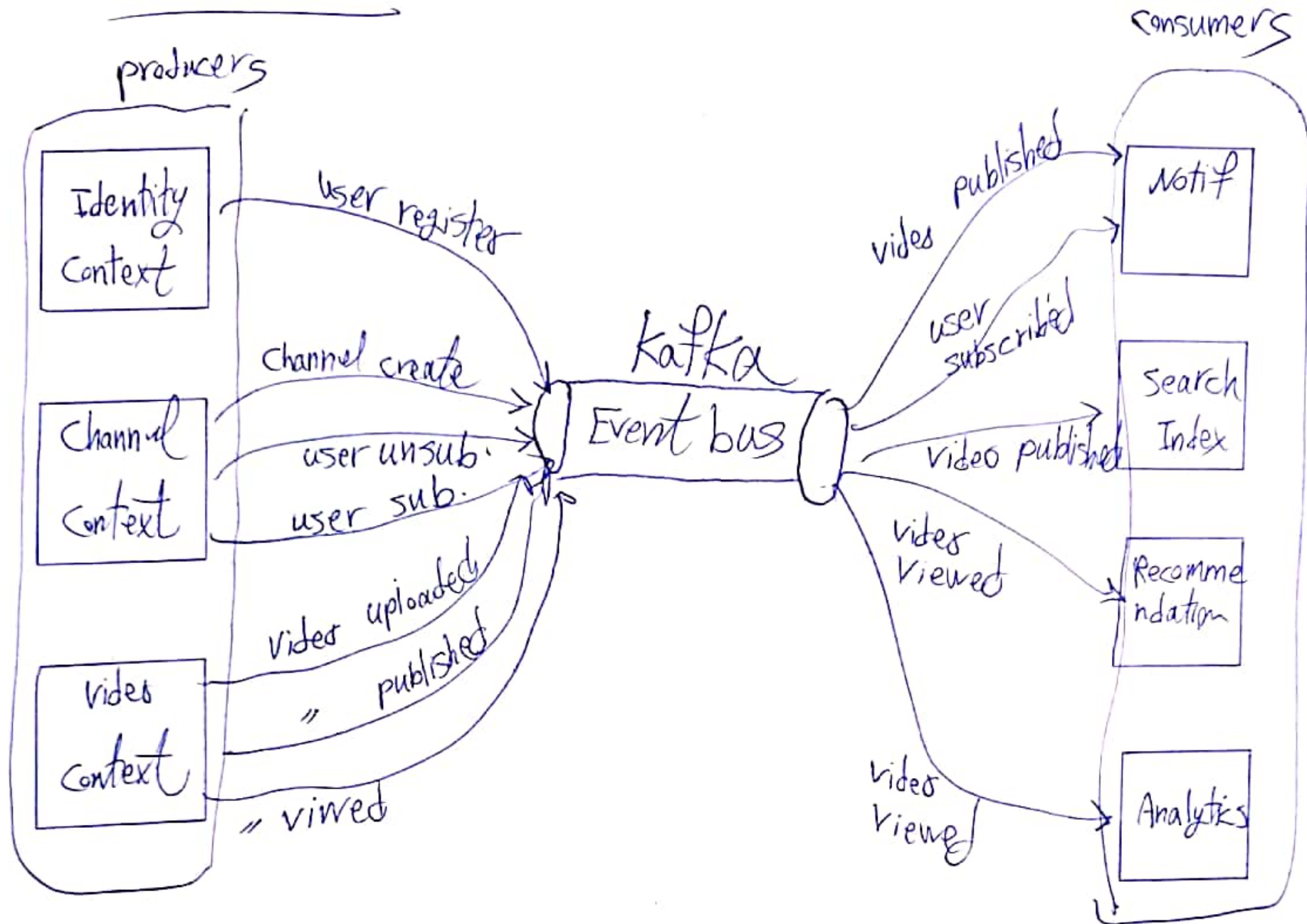




:UML rain juo



Domain Events :



CQRS المبدأ

